



Syscall-BSEM: Behavioral semantics enhancement method of system call sequence for high accurate and robust host intrusion detection

Yifei Zhang, Senlin Luo, Limin Pan*, Hanqing Zhang

Information System and Security & Countermeasures Experimental Center, Beijing Institute of Technology, Beijing 100081, China

ARTICLE INFO

Article history:

Received 18 November 2020

Received in revised form 13 May 2021

Accepted 17 June 2021

Available online 19 June 2021

Keywords:

Host intrusion detection

System call

Obfuscation attack

Behavioral semantics

Deep learning

ABSTRACT

The system call sequence is widely used as raw data due to its prospective performance in host-based intrusion detection methods using machine learning. However, evolutionary intrusion attacks such as the obfuscation technique can achieve the same invasion purpose and effect while changing the malicious system call combination to bypass the abnormal identification, which makes the detection results not robust and even invalid. In this paper, we present a behavioral semantics enhancement method of system call sequence to overcome the problem. This method combines sequence completion to extend behavior information capacity with system calls abstraction and invocation switching differential encoding to improve abstractive representation ability. To complete behavioral semantics features extraction and data classification, the enhanced sequences are transformed to vector matrices and input into the multi-channel Text-CNN. Evaluation experiments show that the proposed method outperforms all of the compared works significantly, which suggests it has a more accurate and robust performance in detecting obfuscation attacks.

© 2021 Elsevier B.V. All rights reserved.

1. Introduction

In cyberspace, all kinds of hosts providing open network services (physical servers, virtual hosts in the cloud, etc.) have become the main targets of intrusion attacks [1,2]. Some hosts are the direct targets of intrusion attacks, some are intermediate springboards for spreading malicious programs, and some are implanted in backdoors as tools for intruders to launch other notorious attacks. These compromised hosts are hijacked to perform different types of attacks such as DDOS (Distributed Denial of Service), ransomware, Trojan and botnet, etc., which pose serious threats to cyberspace security [3].

To defeat these intrusion activities, researchers have developed numerous system calls-based detection methods using machine learning to deal with the above threats. In operating systems, system calls are referred to when a kernel service is requested by a running process, and they are significant interactions between programs and the kernel, which can indicate the host running status [4]. Creech et al. [5] proposed an approach based on natural language processing methodology [6,7], which counts the occurrence frequency of feature phrases extracted by n-grams in each system call sequence as the decisive parameter for calculating feature vectors. Khater et al. [8] applied pattern features obtained by n-grams in system call sequences. The frequency of each pattern feature is normalized by being divided by

the length of the current system call sequence, making the feature vectors more reasonable. There are also many other methods based on features gained by combination rule [9], n-gram [10–13], and subsequence matching frequency [14–16], etc., which are explained in detail in Section 2.

Unfortunately, several evolutionary attacks have been developed [17]. An obfuscation technology proposed by Rosenberg et al. [18] can bypass the system calls-based intrusion detection systems by modifying generated system calls of the malicious code to be classified as benign.

Obviously, the evolutionary intrusion attacks become more diverse and sophisticated. However, those methods mentioned above based on features extracted by combination rule, n-gram, subsequence matching frequency, etc., cannot smoothly handle the challenges to keep detection performance. Because all of these methods require that the constructed features match the identical potential subsequences in each system call trace exactly, which makes the detection methods sensitive and unstable. While after the attacker modifies and replaces these subsequences (e.g., the approach in [18]), the original features will become invalid ultimately, leading to poor performance in detecting evolutionary attacks.

According to the semantic theory of Creech et al. [5], each system call is a basic behavioral semantic unit as a ‘word’, and a traced system call sequence can be viewed as a ‘sentence’ that contains behavioral information of the host running status. Our intuition is that it is nearly unreachable for attackers to radically

* Corresponding author.

E-mail address: panlimin2016@gmail.com (L. Pan).

cloak the malicious behavior and effect of intrusion activities though the obfuscation techniques can modify and replace these subsequences that are the same as the features used by the IDS (Intrusion Detection System) to evade inspections. In order to better cope with the evolutionary intrusion attacks, we believe that by enhancing the behavioral semantic information in the system call sequence and extracting more generalized features for intrusion detection, the anti-attack ability and robustness of the detection method will be improved.

In this paper, we enhance the behavioral semantic information of system call sequences from two aspects: (1) The sequence completion method (detailed in 4.2.1) is used to extend the original system call sequences, which specifically combines sequence repetition and prediction to increase the capacity of behavioral semantic information. (2) The system call abstraction (detailed in 4.2.2) and differential encoding method (detailed in 4.2.3) are used to improve the representation ability of the abstract behavioral semantic information. System calls are divided into different types according to their functions, so system calls with similar behavior can be abstractly represented by their types. The different switching modes of system calls in the sequences are encoded, and then the dynamic changing behaviors of system calls can be abstractly represented by the differential codes.

Different extended sequences generated through the above methods are embedded into vector matrices and then input into a multi-channel CNN (convolutional neural network) to complete feature extraction and data classification. So the multiple behavioral semantics features can be extracted and coalesced through the convolution and pooling layer.

In summary, our main contributions are as follows:

(1) We present a behavioral semantics enhancement method of system call sequence (Syscall-BSEM), which can extend the capacity and improve the abstractive representation ability of behavioral semantic information.

(2) We develop a prototype implementation of the proposed method and conduct comparison experiments to demonstrate the accuracy and robustness advantages.

The rest of this article is organized as follows. Section 2 describes the related works. Section 3 presents the threat model and defence strategies. Section 4 describes the overall framework and its implementation. Section 5 evaluates the performance. Section 6 discusses the proposed method. Finally, Section 7 concludes this paper.

2. Related work

The system calls-based HIDS (Host-based Intrusion Detection System) [19,20] monitors system call traces to detect malicious activities in a specific physical or virtual host. The system calls-based method was firstly proposed by Forrest et al. [21], whose seminal research resulted in many subsequent works [22–24]. The system calls-based HIDS has always gained attention and has been deployed for virtual hosts intrusion detection in the cloud environment [25].

Gupta et al. [9] proposed a structure-based approach for detecting anomalous processes in a private cloud environment. System calls invoked by the benign programs and their immediate following system calls are saved as key–value pairs. If the system call traces monitored in the target VM (virtual machine) cannot match the key–value pairs, the alarm will be triggered. This approach is not suitable for longer sequences and can easily be bypassed by the previously mentioned evasive method [18].

What is more, the frequency of system calls is also commonly used to construct features. Deshpande et al. [14] apply virtual machine system audit log files to extract and calculate the frequency of system calls. All the records have been converted into vectors

representing each process with frequency distribution. A KNN (K-Nearest Neighbor) classifier has been employed to classify the system call traces as it allows easy incorporation of new training documents. Xie et al. [15] provided an initial evaluation of ADFA-LD with experiments using the simple KNN classification and k-means clustering methods. The experimental result shows that frequency-based algorithms save computational resources compared with short sequence-based methods. Xie et al. [16] further improved the performance and reduced the computational cost using a one-class SVM (Support Vector Machine). Liu et al. [26] proposed a statistical pattern-based feature extraction method lately. It first transforms the system call traces into frequency sequences of n-grams and then extracts statistical features of the frequency sequences such as first quartile, second quartile, skewness, etc. These features are built only on the normal data and used to train the one-class classification model for detection. Four different anomaly detection algorithms, LOF (Local Outlier Factor), IF (Isolation Forest), OCSVM (One-Class Support Vector Machine), and KNN, are used to evaluate the stability of the feature extraction method in the experiments. The results indicate their approach is more suitable for anomaly detection across platforms.

The above frequency-based methods and some BoS (Bag of System calls) based methods like Alarifi et al. [27] lose the ordering relationships between system calls which leaves the attacker with a defect that may be evaded. For example, the frequency of each system call in benign trace S_1 : 'open,read,write,close,exit' and abnormal trace S_2 : 'close,open,write,read,exit' are equal, although they have the orders to achieve different intentions.

Similar to NLP (Natural Language Processing) [28], methods such as the n-gram, sliding window algorithm, and TF-IDF (Term Frequency–Inverse Document Frequency) method, etc., have been applied in system calls-based HIDS. Creech et al. [5,10] used multiple-length sliding windows to scan the complete system call trace, and multiple-length n-grams were generated. This method, combined with ELM (Extreme Learning Machine), is proven to effectively increase the detection rate and lower the false alarm [4]. Khater et al. [8] proposed an improved n-gram feature extraction method and a feature selection method based on mutual information. The MLP (Multilayer Perceptron) was used for classification. Borisaniya et al. [11] build a VMI-based (Virtual Machine Introspection based) security framework for the cloud. They derive the process classification features from the extracted in-VM process system call traces and describe them in their modified vector space model using different term-size n-grams. And then, combine the tree-based algorithm J48 (also known as C4.5) to complete intrusion detection. Serpen et al. [13] presented a feature extraction technique based on principal component analysis (PCA) and used the k-nearest neighbor algorithm for classification. The feature vectors are formed by windowing the raw system call traces into fixed-size sequences and the principal component analysis. Mishra et al. [12] proposed VMProtector to extract n-gram features of programs' system call traces inside the virtual machines. VMProtector applies a fusion-based ensemble machine for learning and detecting intrusion attacks. The fusion-based ensemble machine-learning algorithm was composed of RF (Random Forest), KNN and, LR (Logistic Regression). In their subsequent study [29], Mishra et al. implemented KVMInspector for cloud environment. It obtains the programs' system call logs at In-VM and Out-VM level and then takes the 6-grams to extract features. The features are selected by RFE (Recursive Feature Elimination), which reduces the number of features according to their impact on accuracy. It also deploys the ensemble learning algorithm to learn and detect the behavior of benign and malicious programs. In their other research [30], Mishra et al. presented a virtual machine intrusion detection architecture named VM-Guard. It utilizes introspection to capture system call sequences

at the VMM-layer (Virtual Machine Monitor layer). The features of normal and attack sequences are extracted and selected through the proposed ‘BonG (Bag of n-grams)’ approach integrated with the TF-IDF method. Then the Random Forest is used to classify different categories of intrusions in the monitored virtual machine. Recently, Subba et al. [31] proposed a HIDS framework based on the tfidfvectorizer and truncated singular value decomposition (SVD). The framework transforms the system call trace files to n-gram feature vectors. And then, the tfidfvectorizer recognizes the more significant n-gram terms for effective classification. Moreover, the dimensionality of transformed n-gram feature vectors is reduced by truncated SVD, which decreases the processing overhead. Finally, the dimensionality decreased tfidfvectorized feature vectors are input to Neural Network, Decision Tree, and SVM to carry out the classification.

Although these methods all use the n-gram language model for feature construction and extraction, they focus on the combinations of system calls in different semantic structures rather than the behavioral semantic meaning of system calls. The performance is determined by the n-gram features matching exactly, so it is very sensitive to the changes in system call sequences. When attackers use obfuscation techniques [18] to modify system call sequences generated by the malicious programs, the detection capabilities of these methods will be significantly reduced.

Overall, these above detection methods based on features such as combination structures, n-gram subsequences and features matching frequencies, etc., cannot deal with the evolutionary attacks effectively.

This paper proposes a behavioral semantic enhancement method for system call sequences, which can extend the capacity and improve the abstract representation ability of behavioral semantic information of host running activities (including benign and intrusive). This method enables the detection system to extract the attack behavior features even if the obfuscation attacks add or change the system calls in the traces sequences. In addition, our detection architecture (detailed in Section 4) combines the word2vec to generate the embedding vectors with the triple channels Text-CNN to work as the features extractor and data classifier, which can capture multiple fusion features of behavioral semantic information.

3. The threat model and defence strategy

In this section, a brief illustration of the system calls-based HIDS attack paths is given. The defence strategies related to the design of the proposed method are introduced.

3.1. Threat model

In this paper, evolutionary attack refers to the obfuscation technique. It changes the system call traces in the victim host caused by the attack on the premise of realizing the original intrusion target and attack effect, so as to bypass and evade the detection of system calls-based HIDS.

The attack paths are illustrated in Fig. 1, there are two main attack paths:

(1) The No.1 attack path generates a system call sequence different from the one that was originally classified as malicious by changing the system calls invoked by the running intrusion malware. There are two methods to realize the attack path. The first is to insert ‘no-ops’ into the system call trace by modifying the malware code to call ‘no-op’ system calls. A ‘no-op’ indicates a system call with no actual effect or whose effect is unrelated to the attacker’s purpose. The second is equivalent attack, which is to modify the malware code to achieve the original malicious attack effect while calling the system calls of different combined

forms to generate the trace sequence different from the original malicious sequence. There are often many different combinations of system calls that can accomplish the same function.

(2) The No.2 attack path generates a system call sequence different from the one that was initially classified as malicious by modifying the system calls generated in the victim host caused by the network intrusion attack. The attack path is realized by using different network attack vectors with the same attack effect to make the victim host produce other sequences different from the original malicious subsequence.

3.2. Defence strategy

In order to counter the above attacks, we reiterate the defence strategy followed by the method presented in this paper:

(1) Extending the capacity of the behavioral semantic information in system call sequences.

(2) Improving the representation ability of abstract behavioral semantic information in system call sequences.

Strategy 1 is implemented by using the sequence completion method, which includes sequence repetition and sequence prediction:

In the sequence prediction part, the RNN (Recurrent Neural Network) based seq2seq model [32,33] is used in this paper, considering it can aggregate the information in the source sequence (original sequence) and affect the generation of the target sequence (prediction sequence). After the obfuscation attack modifies the feature subsequence in the source sequence, the seq2seq model can still capture the context information in the abnormal source sequence. The predicted target sequence may still contain abnormal components. The sequence prediction based on seq2seq is combined with sequence repetition to fill the raw system call sequences, while the sequence repetition is used to preserve the original information of intrusion behavior.

This paper realizes strategy 2 by extending the behavioral semantics representation, which includes system call abstraction and differential encoding:

(1) Considering the significant number of system call combinations and the frequent changes in new attack patterns, we propose the system call abstraction by adopting their types. For example, the original malicious system call combination is ‘syscall₁ syscall₂ syscall₃’, and the type mode is ‘type₁ type₂ type₃’. The attacker may use a ‘syscall_x’ with similar functions to replace ‘syscall₂’, but ‘syscall_x’ and ‘syscall₂’ may all belong to the ‘type₂’. So the attack can still be detected by the type mode. Compared to the specific system calls directly, using system call types can capture the abstract behavioral information of abnormal activities to make the anomaly identification more stable and resilient against obfuscation attacks.

(2) Our research found a difference in the switching behavior of system calls between attack traces and normal traces. Attack system call traces are switched more frequently than normal sequences. We invent a novel ‘differential encoding’ technology to enhance the representation of invocation switching features in the system call sequence. It is the first time to use the system calls’ continuous changing relationships to characterize the different system calls switching mode of normal and abnormal programs to the best of our knowledge.

4. Proposed method

4.1. The framework

This section describes the overall architecture of our host intrusion detection system based on the behavioral semantics

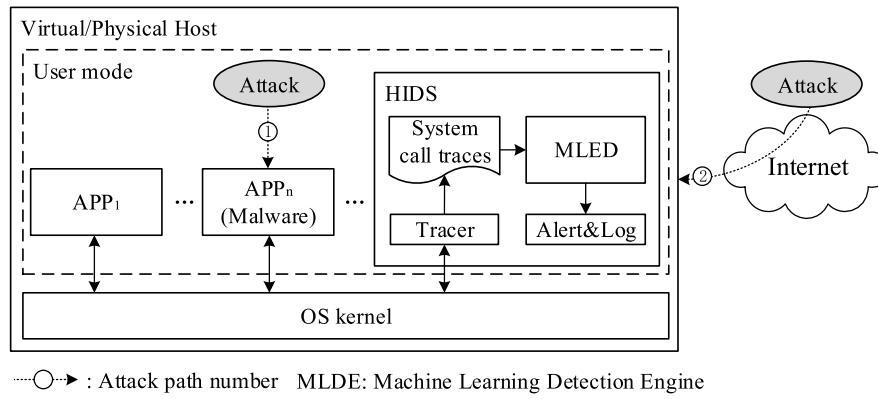


Fig. 1. System calls-based HIDS attack paths.

enhancement method. The framework is shown in Fig. 2. It inputs system call sequences and outputs the classification results that indicate whether the input sequences are normal or abnormal. It consists of three modules: behavioral semantics enhancement module, extended sequences vectorization module, feature extraction and data classification module.

(1) Behavioral semantics enhancement module.

This module gains the unified length extended sequences of the input system call traces through sequence completion (described in 4.2.1). Then it replaces the specified system call numbers by the types defined in this paper (detailed in 4.2.2) to create the type extended sequences. On the other hand, it encodes the unified length sequences to generate the code extended sequences via differential encoding (illustrated in 4.2.3). In the end, this module forms three kinds of extended sequences containing different features for each input system call trace.

(2) Extended sequences vectorization module.

This module leverages the word segmentation and pre-training technology in the natural language processing field to gain token sequences of extended sequences and embed each token of these sequences into its vector representation via ‘Syscall2Vec’, ‘SyscallType2Vec’, and ‘SyscallCode2Vec’ based on the word2vec model [34] (detailed in 4.3). At last, this module forms the numerical matrix representation for three kinds of extended sequences, respectively.

(3) Feature extraction and data classification module.

This module integrates the Text-CNN to fuse and extract high-dimensional features of intrusion behaviors in extended sequences. It takes the representation matrices of extended sequences as its input, and they are input into the network through channel-1, 2, 3 individually. Finally, the classification results determine whether the input sequences are normal or abnormal.

The workflow of the proposed HIDS based on Syscall-BSEM is shown in Fig. 3. The input of this HIDS is system call sequences. First, it generates three kinds of extended sequences based on the inputs through behavioral semantics enhancement. Next, it transforms each extended sequence into a token sequence via sequence segmentation and vectorizes each extended sequence represented as a token sequence into a matrix representation. Then, the representation matrices corresponding to the input system call sequences are obtained, and they are split into training data and testing data. Finally, it uses the training data to train the Text-CNN, and the testing data is fed into the trained Text-CNN to output the classification results.

4.2. Behavioral semantics enhancement module

The original intention of behavioral semantics enhancement is to express high-dimensional abstract features in the system

call sequence and increase the discrimination of different types of sequences, thereby improving the intrusion detection system’s performance in this paper. The workflow of Syscall-BSEM is shown in Fig. 4. It contains three primary operations: sequence completion, system call abstraction, and differential encoding. Firstly, it fills the input system call sequences to gain the unified length sequences by repetition and prediction of sequence completion. Then it leverages the 63 defined types to abstract system calls of the unified length sequences, which creates the type extended sequences. On the other hand, it adopts four codes generated by differential encoding to transform the unified length sequences to the code extended sequences.

The inputs of our intrusion detection system are system call sequences composed of system call numbers. A section of the raw system call sequence is intercepted, and its data format is shown in Fig. 5. There is a space between each two adjacent system call numbers.

System call sequences similar to that shown in Fig. 5 can be expressed as the algebraic representation shown in Fig. 6. In our method, we use a more general form to represent the input data. We denote the system call sequence as Seq_i with $i \in [1, L]$ and use $\{C_{ij} \in \phi | i, j \in N^*\}$ to describe the system call number, where C_{ij} represents the j th system call number of the i th input sequence, and ϕ represents the set of all system call numbers. This facilitates the usage of the unified term in the following description.

4.2.1. Sequence completion

This paper considers extending the sequence length and hopes to expand the information capacity based on retaining the original sequence information. This section introduces the sequence completion method. The main idea of this method is to combine sequence repeating with sequence predicting to extend the input sequence to a unified length. We believe that repeating sequences can retain the semantic features of sequence to the greatest extent while predicting sequences can increase the information capacity.

We need to determine the unified length of completion sequences first. The choice of suitable extension length is a balanced result. In this paper, Text-CNN is used for feature extraction, and the network uniformly adjusts the length of the input sequence to a reference value that can be set. When the input sequence length is less than this reference value, the user can supplement the sequence with custom data. When the input sequence length is bigger than this reference value, the sequence will be truncated, obviously damaging or even losing the original sample information. At the same time, a larger reference value will increase the workload of the Text-CNN. Therefore, we choose the maximum length of all sample sequences as the reference value, which can

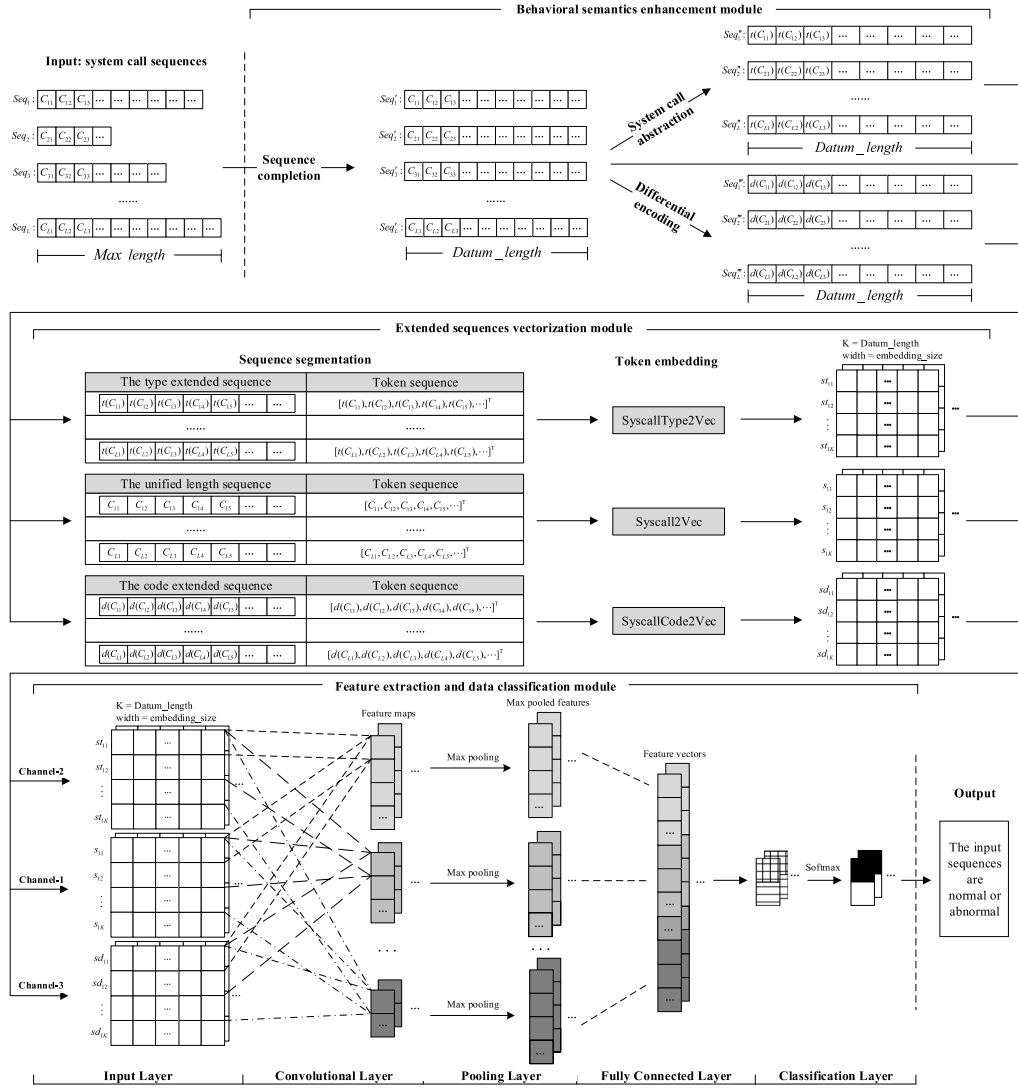


Fig. 2. The framework of the proposed HIDS.

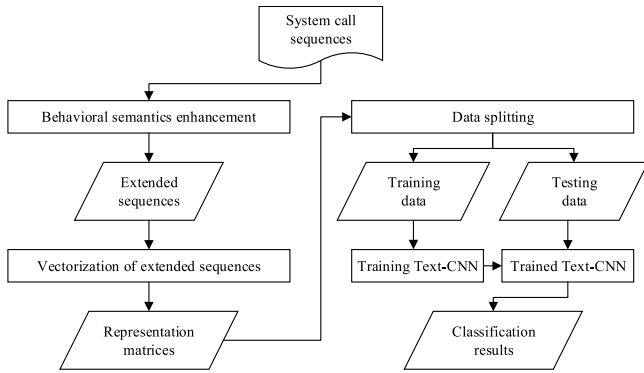


Fig. 3. The workflow of the proposed HIDS.

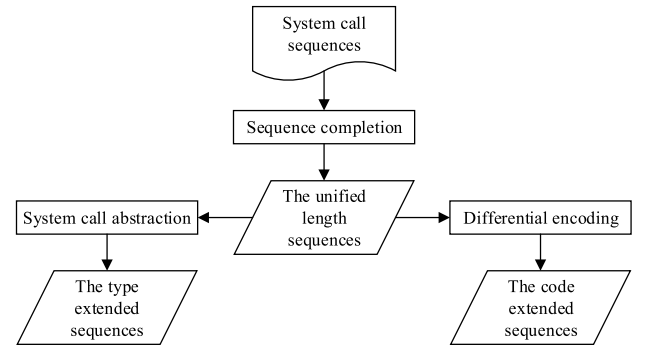


Fig. 4. The workflow of the behavioral semantics enhancement method.

6 6 63 6 42 120 6 195 120 6 6 114 114 1 1 252 252 252

Fig. 5. The data format of a raw system call sequence.

meet the requirements of preserving and expanding sequence information and give consideration to the efficiency of Text-CNN.

The unified length is defined as *Datum_length*, and its theoretical value range is shown in Eq. (1).

$$Datum_length \geq \text{MAX}(\text{length}(\text{Seq}_i)), i \in [1, L] \quad (1)$$

The $\text{MAX}(\cdot)$ operator is used to calculate the maximum sequence length, and the $\text{length}(\cdot)$ operator is used to calculate the sequence length. We use the minimum unified length, i.e., the maximum length of all input sequences. The reason is given above.

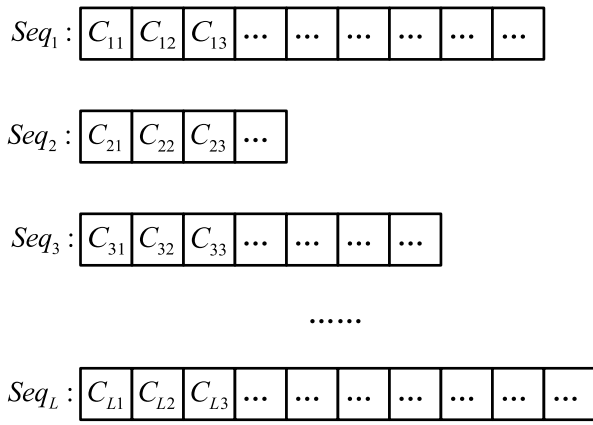


Fig. 6. The algebraic representation of system call sequences.

For an input sequence Seq_i , our method determines the sequence repeating times R_i based on the $length(Seq_i)$ and $Datum_length$. The calculation formula is shown in Eq. (2), and the $INTEGER(\cdot)$ operator performs a rounding down operation.

$$R_i = \text{INTEGER}\left(\frac{Datum_length}{length(Seq_i)}\right) \quad (2)$$

When a specific input sequence Seq_i is filled to the unified length, the rounding down operation is used to determine the repeating times. It is easy to think that even if the input sequence is repeated R_i times, it may still not reach the unified length, so we need to generate a sequence with a length between 1 and $length(Seq_i)$ to complete the rest. When the $Datum_length$ and R_i are determined, the length G_i of the remaining sequence used to complete the rest will be determined by Eq. (3).

$$G_i = Datum_length - length(Seq_i) \times R_i \quad (3)$$

For how to generate this G_i length sequence, one of the available methods is to intercept the continuous subsequence of input sequence Seq_i with the length of G_i . However, this method may truncate some continuous subsequence patterns in the raw sequence Seq_i , thus losing some behavioral semantics information. In natural language processing, two articles [32,33] in 2014 independently elaborated and proposed the seq2seq model, which is mainly used in machine translation, chat robot, syntax analysis, text summarization, and other fields. Inspired by the seq2seq model, we complement the input sequence by using it to generate a prediction sequence of a specified length.

We use replication to repeat the sequence. So it is simple and will not be described in detail. Here we mainly introduce the generation of predictive sequences based on the seq2seq model. This model is an RNN implementation of an Encoder-Decoder network structure [35], and its input and output are all sequences. The encoder converts the context information of a variable-length sequence into a fixed-length vector. In contrast, the decoder converts this fixed-length vector containing context information of the input sequence into a variable-length target sequence. This design is in line with our needs. We want to get a prediction sequence based on the context semantics information of system calls in the input sequence Seq_i and use this prediction sequence to complement the sequence to the unified length.

The sequences prediction model called ‘system-calls to system-calls’ in this paper is shown in Fig. 7. This architecture is inspired by the seq2seq model. This model reads an input system call sequence ‘ $C_{i1} C_{i2} \dots C_{in}$ ’ and predicts ‘ $C_{i(n+1)} C_{i(n+2)} \dots C_{i(n+m)}$ ’ as the output system call sequence. This model stops predicting after outputting the end-of-sequence (EOS) token.

According to the principle and processing of the seq2seq model [32], for a given system call sequence Seq_i the encoder will complete the forward propagation calculation to generate the hidden state and the output at that moment. The hidden state at the last moment is represented as the context information vector of Seq_i , and this vector will be used as the initial hidden state of the decoder. The output of the decoder at each moment will be affected by the input sequence information carried by this vector. Each output generated by the decoder requires the participation of the hidden state and the output at the previous moment.

Through sequence completion, we can gain unified length sequences based on the input from Fig. 6. As shown in Fig. 8, Seq'_L represents the L th unified length sequence which is made up of $\{C_{ij} | i = L, j \in [1, Datum_length]\}$. These unified length sequences retain original features of input sequences and expand the prediction behavior information.

The algorithm of sequence completion is summarized in Algorithm 1.

In the field of natural language processing, the Long Short-Term Memory (LSTM) has been proven to be good at handling long-term dependent sequences [36]. Therefore, our proposed method uses the LSTM model, and we need to use the corresponding pairs of data (called source sequence and target sequence) for training. We divide each system call sequence in the dataset into several fixed-length subsequences in order. These front subsequences are used as the source sequences, and the next subsequences following the front ones are used as the target sequences. The source sequence and target sequence pairs are used to train the system-calls to system-calls model.

Algorithm 1 : The sequence completion algorithm

Input: System call sequences: Seq_i ($i = 1, 2, \dots, L; L \in N^*$);
Initialization: $Datum_length = \text{MAX}(length(Seq_i))$;
1: $seq_generator()$ is a function of the trained seq2seq model to generate prediction sequence;
2: Seq_i is a list of system calls;
3: **for** $i = 1$ to L **do**
4: $seq_len = length(Seq_i)$;
5: $R_i = \lfloor Datum_length / seq_len \rfloor$;
6: $G_i = Datum_length \% seq_len$;
7: $seq_source = Seq_i[(seq_len - G_i) : (seq_len - 1)]$;
8: $seq_generation = seq_generator(seq_source)$;
9: **if** $length(seq_generation) < G_i$ **then**
10: $sub_repeating_times = \lfloor G_i / seq_generation \rfloor$;
11: $padding_len = G_i \% length(seq_generation)$;
12: $sub_padding_list = seq_generation[0 : (padding_len - 1)]$;
13: $seq_generation = (seq_generation * sub_repeating_times) + sub_padding_list$;
14: **end if**
15: **if** $length(seq_generation) > G_i$ **then**
16: $seq_generation = seq_generation[0 : (G_i - 1)]$;
17: **end if**
18: $Seq'_i = (Seq_i * R_i) + seq_generation$;
19: **end for**
Output: Unified length sequences: Seq'_i ($i = 1, 2, \dots, L; L \in N^*$).

4.2.2. System call abstraction

In various operating systems (Linux, Windows, Mac, etc.), the kernel provides a series of system calls. The operating system assigns a unique number to each system call function and stores mapping relationships between numbers and system calls in the specific system files. For example, the mapping relationship of the Linux system is stored in the ‘/usr/include/asm/unistd.h’ file. According to different functions, these system calls can be roughly

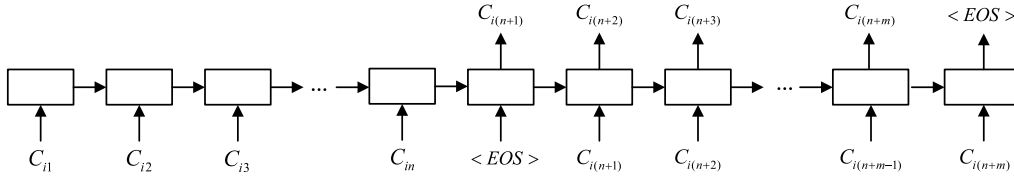


Fig. 7. The system-calls to system-calls model.

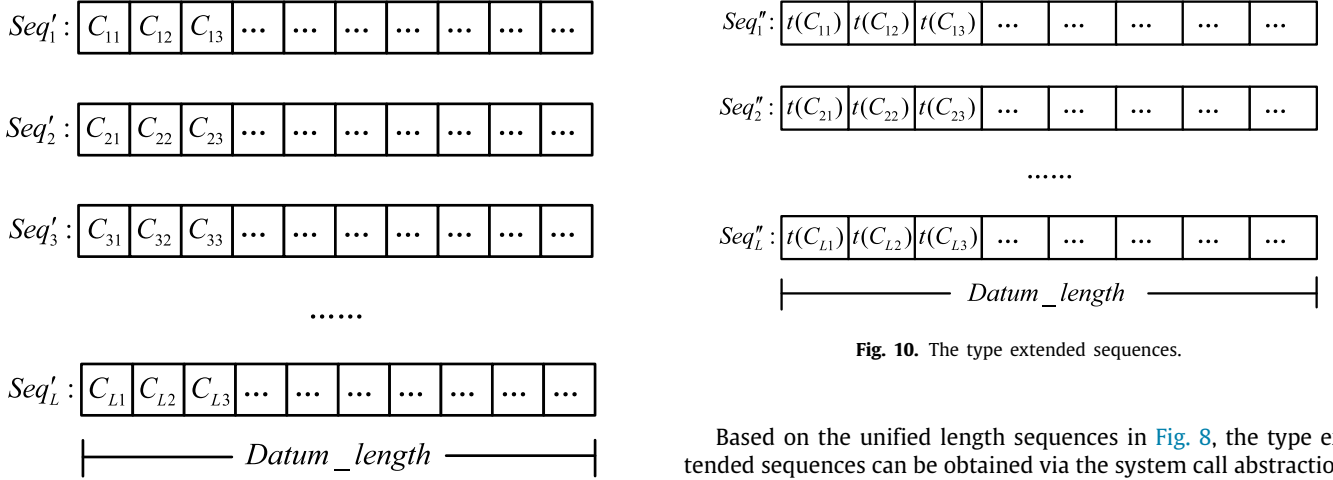


Fig. 8. The completion sequences with unified length.

```

/* fs/open.c */
#define __NR3264_statfs 43
__SC_3264(__NR3264_statfs, sys_statfs64, sys_statfs)
#define __NR3264_fstatfs 44
__SC_3264(__NR3264_fstatfs, sys_fstatfs64, sys_fstatfs)
#define __NR3264_truncate 45
__SC_3264(__NR3264_truncate, sys_truncate64, sys_truncate)
#define __NR3264_ftruncate 46
__SC_3264(__NR3264_ftruncate, sys_ftruncate64, sys_ftruncate)

```

Fig. 9. The mapping relationship between system call numbers and functions.

divided into seven categories: process control, file system control, system control, memory management, network management, user management, and inter processes communication.

The file containing the detailed system call definition information corresponding to the system call numbers in the ADFA-LD dataset [37] is used to define the system call types. The mapping relationship between system call numbers and system call functions in this file is shown in Fig. 9. Based on the seven categories listed above, combined with the detailed information provided in this file, we conduct type mapping for all system call numbers. For example, from the contents of this file, it can be seen that the system calls '43', '44', '45', and '46' are related to the open operation controlled by the file system.

Depending on the above information and the comments in the mapping relationship file, we define the type of system call numbers '43-46' as 'fs_open', 'fs' indicates that this type belongs to the file system control category, and 'open' denotes the open operation in the file control system. In this way, we get 63 types, as shown in Table 1.

Then we define operator $Type(\cdot)$ to match the type of system call and use ψ to represent the set of 63 system call types. So the types of system calls can be represented by $\{Type(C_{ij}) \in \psi \mid i, j \in N^*\}$. Therefore, we define the system call abstraction as the function $t(\cdot)$, shown in Eq. (4).

$$t(C_{ij}) = Type(C_{ij}) \quad (4)$$

Fig. 10. The type extended sequences.

Based on the unified length sequences in Fig. 8, the type extended sequences can be obtained via the system call abstraction. As displayed in Fig. 10, Seq''_L represents the L th type extended sequence which consists of $\{t(C_{ij}) \mid i = L, j \in [1, Datum_length]\}$.

The system call abstraction algorithm is shown in Algorithm 2.

Algorithm 2 : The system call abstraction algorithm

Input: Unified length sequences: Seq'_i ($i = 1, 2, \dots, L$; $L \in N^*$);

Initialization: $\psi = \{type_1, \dots, type_{63}\}$: the set of system call types;

1: Seq'_i is a list of system calls;

2: **for** $i = 1$ to L **do**

3: $seq_len = length(Seq'_i)$;

4: **for** $j = 0$ to $(seq_len - 1)$ **do**

5: $Seq''_i[j] = t(Seq'_i[j])$;

6: **end for**

7: **end for**

Output: Type extended sequences: Seq''_i ($i = 1, 2, \dots, L$; $L \in N^*$).

4.2.3. Differential encoding

When the host is running normally, the processes and state of the operating system are regular and stable. When the host is invaded, such a stable state will be broken. For example, when an intruder uses a remote brute force attack to crack the computer's login password, the operating system frequently rejects the login behavior because the password used by the intruder is wrong in most cases. This dynamic behavior of the operating system is manifested at the kernel level by constantly switching different system calls. The running state of the operating system is no longer regular and stable.

Based on the above analysis, we have reason to believe that the changes in the operational process and running status of the operating system before and after the host computer under attack can be represented by changes in system calls. However, the sequences formed by tracking the system calls are single in content and simple in structure. Although it contains the characteristics of switching mode, it is not fully and accurately expressed. Therefore, we propose a 'differential encoding' method for describing

Table 1
The abstraction types of system calls.

Function category	Type		
Process control	kernel_acct	kernel_fork	kernel_sched
	kernel_capability	kernel_futex	kernel_signal
	kernel_exec_domain	kernel_hrtimer	wait4
	kernel_exit	kernel_itimer	
File system control	fs_xattr	fs_locks	fs_sendfile
	fs_dcache	fs_namei	fs_select
	fs_cookies	fs_namespace	fs_signalfd
	fs_eventfd	fs_nfsctl	fs_splice
	fs_eventpoll	fs_open	fs_stat
	fs_fcntl	fs_pipe	fs_sync
	fs_inotify_user	fs_quota	fs_timerfd
	fs_ioctl	fs_readdir	fs_utimes
	fs_ioprio	fs_read_write	
System control	kernel_kexec	kernel_time	prlimit64
	kernel_module	kernel_timer	fanotify_init
	kernel_posix-timers	security_keys_keyctl	fanotify_mark
	kernel_printk	kernel_sys_example	syscalls
	kernel_ptrace	arch_specific_syscall	io
Memory management	mm_filemap	mm	mm_nommu
	mm_fadvise		
Network management	net_socket		
User management	kernel_sys		
Inter processes communication	ipc_mqueue	ipc_shm	ipc_msg
	ipc_sem		
Else	others		

the switching mode in the system call sequence. This method introduces the idea of differential encoding approaches in the field of information and communication technology. The system call is regarded as a positive integer value that can be compared in size. The deviation between the current system call value and the next in the sequence represents the characteristics of switching mode. We define the encoding forms of differential results to accurately characterize various changing ways of a system call in the sequence.

First, for a current system call number C_{ij} in the system call sequence Seq_i , we define four codes with different meanings, as shown in the following Eq. (5).

$$\Delta(C_{ij}) = \begin{cases} \delta_0 = 00, & C_{ij}^{int} = C_{i(j+1)}^{int} \\ \delta_1 = 01, & C_{ij}^{int} < C_{i(j+1)}^{int} \\ \delta_2 = 10, & C_{ij}^{int} > C_{i(j+1)}^{int} \\ \delta_3 = 11, & C_{i(j+1)} = \emptyset \end{cases} \quad (5)$$

The $\Delta(\cdot)$ operator represents the encoding of the current system call. The encodings are the binary representations of the decimal numbers 0, 1, 2, and 3. The C_{ij}^{int} and $C_{i(j+1)}^{int}$ are integer values of C_{ij} and $C_{i(j+1)}$. The δ_0 in Eq. (5) represents the value C_{ij}^{int} of the current system call C_{ij} is equal to $C_{i(j+1)}^{int}$ of the following system call $C_{i(j+1)}$, which means the current system call has not switched. While the δ_1 and δ_2 indicate the value of the current system call C_{ij} is different from the following system call $C_{i(j+1)}$, which means the system call has switched. The δ_3 represents the current system call C_{ij} is the end of the sequence.

Then, we define the differential encoding of system calls as the function $d(\cdot)$. The function is shown in Eq. (6) and $\oplus$ represents the concatenation of characters.

$$d(C_{ij}) = C_{ij} \oplus \Delta(C_{ij}) \quad (6)$$

We can generate the code extended sequences based on the completion sequences of unified length in Fig. 8. As shown in Fig. 11, Seq_L'' represents the L th code extended sequence which is composed of $\{d(C_{ij}) | i = L, j \in [1, Datum_length]\}$.

The differential encoding algorithm can be seen in Algorithm 3.

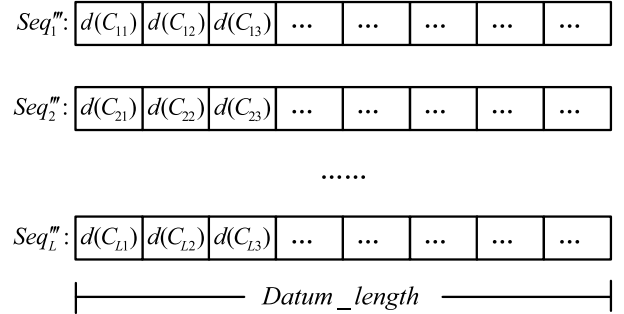


Fig. 11. The code extended sequences.

Algorithm 3 : The differential encoding algorithm

Input: Unified length sequences: Seq_i' ($i = 1, 2, \dots, L; L \in N^*$);
Initialization: $\Delta = \{\delta_0, \delta_1, \delta_2, \delta_3\}$: the set of encodings;
1: Seq_i' is a list of system calls;
2: **for** $i = 1$ to L **do**
3: $seq_len = \text{length}(Seq_i')$;
4: **for** $j = 0$ to $(seq_len - 1)$ **do**
5: **if** $j = (seq_len - 1)$ **then**
6: $Seq_i''[j] = Seq_i'[j] \oplus \delta_3$;
7: **end if**
8: **if** $Seq_i'[j]^{int} = Seq_i'[j+1]^{int}$ **then**
9: $Seq_i''[j] = Seq_i'[j] \oplus \delta_0$;
10: **end if**
11: **if** $Seq_i'[j]^{int} < Seq_i'[j+1]^{int}$ **then**
12: $Seq_i''[j] = Seq_i'[j] \oplus \delta_1$;
13: **end if**
14: **if** $Seq_i'[j]^{int} > Seq_i'[j+1]^{int}$ **then**
15: $Seq_i''[j] = Seq_i'[j] \oplus \delta_2$;
16: **end if**
17: **end for**
18: **end for**

Output: Code extended sequences: Seq_i''' ($i = 1, 2, \dots, L$; $L \in N^*$).

4.3. Extended sequences vectorization module

This module is responsible for embedding three extended sequences of each input system call trace into their numerical matrices representation.

This processing is the premise of using deep learning neural networks to extract features automatically, as shown in Fig. 12. The first step is sequence segmentation, like word segmentation of NLP, i.e., transforming each extended sequence into a token sequence. The sequence segmentation rule is defined as simply splitting by element. We denote the token sequences as $\{T_i', T_i'', T_i'''; i \in [1, L]\}$ (shown in Eq. (7), Eq. (8), Eq. (9)). They are corresponding to the extended sequences $\{Seq_i', Seq_i'', Seq_i'''; i \in [1, L]\}$.

$$T_i' = [C_{i1}, C_{i2}, C_{i3}, \dots, C_{iDatum_length}]^T, i \in [1, L] \quad (7)$$

$$T_i'' = [t(C_{i1}), t(C_{i2}), t(C_{i3}), \dots, t(C_{iDatum_length})]^T, i \in [1, L] \quad (8)$$

$$T_i''' = [d(C_{i1}), d(C_{i2}), d(C_{i3}), \dots, d(C_{iDatum_length})]^T, i \in [1, L] \quad (9)$$

Given the token sequences, this module embeds tokens C_{ij} , $t(C_{ij})$, $d(C_{ij})$ ($i \in [1, L]$, $j \in [1, Datum_length]$) into vectors s_{ij} , st_{ij} , sd_{ij} ($i \in [1, L]$, $j \in [1, Datum_length]$) through the word2vec [34] based models (i.e., Syscall2Vec, SyscallType2Vec, and SyscallCode2Vec). These output vectors finally form the matrices for the extended sequences. We denote the output matrices of the unified length sequences, the type extended sequences, and the code extended sequences as S_i , ST_i , SD_i with $i \in [1, L]$ (shown in Eq. (10), Eq. (11), Eq. (12)). The height of these matrices is $Datum_length$, and the width is the *embedding_size* of vectors s_{ij} , st_{ij} , sd_{ij} .

$$S_i = [s_{i1}, s_{i2}, s_{i3}, \dots, s_{iDatum_length}]^T, i \in [1, L] \quad (10)$$

$$ST_i = [st_{i1}, st_{i2}, st_{i3}, \dots, st_{iDatum_length}]^T, i \in [1, L] \quad (11)$$

$$SD_i = [sd_{i1}, sd_{i2}, sd_{i3}, \dots, sd_{iDatum_length}]^T, i \in [1, L] \quad (12)$$

The word2vec is a specific neural network designed for generating distributed vector representations of words. It takes the word corpus as input and uses the context information to train the representations. Since the vector representation incorporates some basic contextual information, the word2vec model can make the vectors of different tokens with similar contexts closer in the representation space. There are two ways to train the word2vec, i.e., CBOW (continuous bag-of-words) and Skip-gram [38,39]. In this work, we adopt the Skip-gram method and construct three corpora based on all of the raw input system call sequences in the dataset and their extended sequences obtained via the same operations in Sections 4.2.2 and 4.2.3 to train our word2vec based models (i.e., Syscall2Vec, SyscallType2Vec, and SyscallCode2Vec). It should be noted that the output extended sequences of the behavioral semantics enhancement module in Section 4.2 are not directly used as the corpora.

The extended sequences vectorization algorithm is shown in Algorithm 4.

Algorithm 4 : The extended sequences vectorization algorithm

Input: The extended sequences: $\{Seq_i', Seq_i'', Seq_i'''; i \in [1, L]\}$;

1: *Segmentation()* is a function to generate token sequence;

2: *Syscall2Vec()*, *SyscallType2Vec()*, *SyscallCode2Vec()* are functions of the trained word2vec models to generate vectors for tokens;

3: **for** $i = 1$ to L **do**

4: $T_i' = \text{Segmentation}(Seq_i')$;

5: $T_i'' = \text{Segmentation}(Seq_i'')$;

6: $T_i''' = \text{Segmentation}(Seq_i''')$;

7: **for** $j = 1$ to $Datum_length$ **do**

8: $s_{ij} = \text{Syscall2Vec}(C_{ij})$;

9: $st_{ij} = \text{SyscallType2Vec}(t(C_{ij}))$;

10: $sd_{ij} = \text{SyscallCode2Vec}(d(C_{ij}))$;

11: **end for**

12: $S_i = [s_{i1}, s_{i2}, s_{i3}, \dots, s_{iDatum_length}]^T$;

13: $ST_i = [st_{i1}, st_{i2}, st_{i3}, \dots, st_{iDatum_length}]^T$;

14: $SD_i = [sd_{i1}, sd_{i2}, sd_{i3}, \dots, sd_{iDatum_length}]^T$;

15: **end for**

Output: The representation matrices: $\{S_i, ST_i, SD_i | i \in [1, L]\}$.

4.4. Feature extraction and data classification module

To make sure the deep learning neural network can fully extract the behavioral semantics features in the three kinds of extended sequences of each input system call trace, we need to input all the representation matrices of extended sequences into the network. We built the feature extraction and data classification module integrated with the triple-channel Text-CNN to achieve this goal. The design is partially inspired by Gu et al. [40] and Kim et al. [41].

Fig. 13 describes the overall structure of this module, and we further illustrate it in detail below. The structure of this module includes an input layer, a convolutional layer, a pooling layer, a fully connected layer, and a classification layer. This module takes representation matrices S_i , ST_i , SD_i with $i \in [1, L]$ as the inputs of channel-1, channel-2, and channel-3. The triple-channel Text-CNN is adopted for features extraction, and its three input channels are used to accept three representation matrices corresponding to each input system call sequence simultaneously. The convolution layer performs convolution operations on the three matrices through convolution filters each time. When performing the convolution operation on the matrices from three channels, the size of each filter is set to $h \times embedding_size \times 3$, where h , $embedding_size$, and 3 denote filter height, filter width, and filter depth, respectively. The filter height refers to the number of vectors in one matrix covered by the convolution filter window in each convolution calculation. The filter width is fixed to the width of vectors s_{ij} , st_{ij} , sd_{ij} ($i \in [1, L]$, $j \in [1, Datum_length]$) so that the convolution only runs on one dimension. And the filter depth is the number of input channels. The convolution operation can be expressed by Eq. (13).

$$c_t = f(W_1 \cdot S_{i,t:t+h-1} + W_2 \cdot ST_{i,t:t+h-1} + W_3 \cdot SD_{i,t:t+h-1} + b) \quad (13)$$

In the above formula, c_t represents a single feature value obtained by convolution operation, $f(\cdot)$ represents the activation function ReLU (Rectified Linear Unit), and $W_1, W_2, W_3 \in \mathbb{R}^{h \times embedding_size}$ represent the weight parameter matrices of a convolution filter on the channel-1, channel-2, channel-3, respectively. Each time the convolution filter passes through an input volume in the size of $h \times embedding_size \times 3$, it will get a feature value. The vectors from t to $(t + h - 1)$ in the matrices of three different extended sequences are represented by $S_{i,t:t+h-1}$, $ST_{i,t:t+h-1}$, $SD_{i,t:t+h-1}$ and b represents the bias in the convolutional layer. After each filter finishes performing convolution operation on all input matrices from the three channels, it outputs a feature map c , which is shown in Eq. (14),

$$c = (c_1, c_2, \dots, c_{Datum_length-h+1}) \quad (14)$$

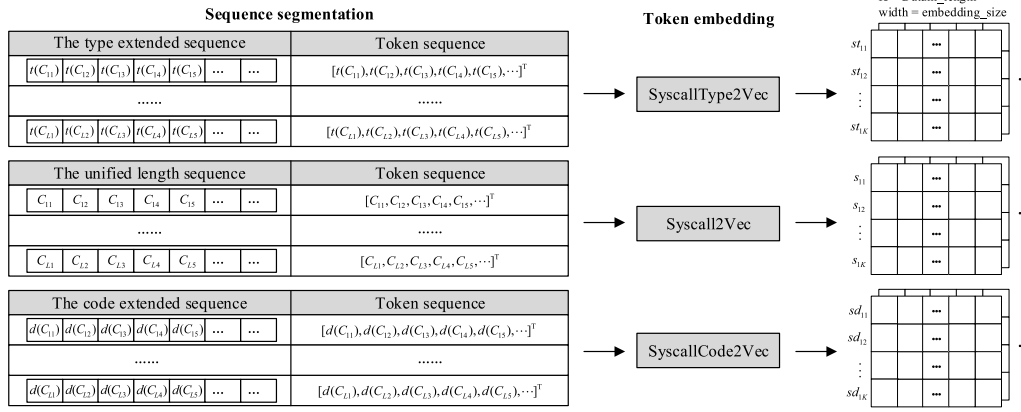


Fig. 12. The extended sequences vectorization module.

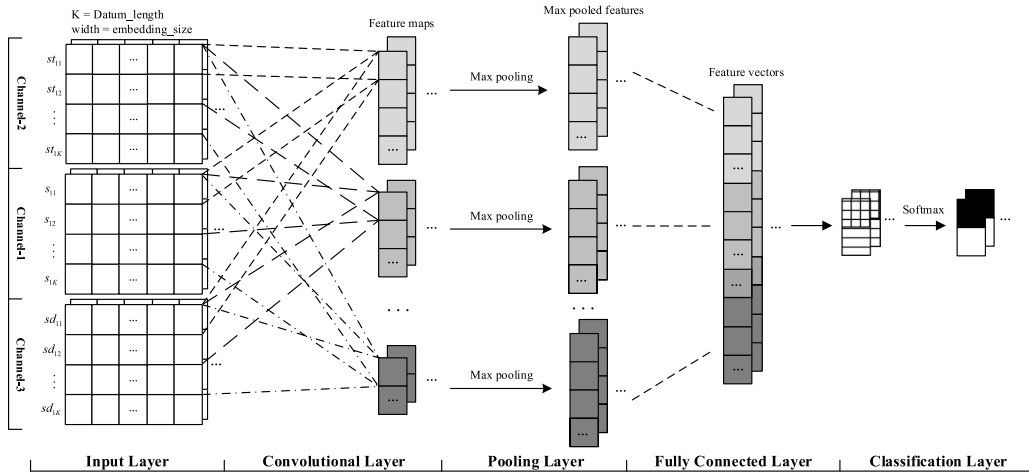


Fig. 13. The feature extraction and data classification module.

Table 2
Statistics of the ADFA-LD dataset.

No.	Data type	Number of traces	Class of traces
1	TRAINING DATA MASTER (TDM)	833	Normal
2	ATTACK DATA MASTER (ADM)	746	Abnormal

where *Datum_length* represents the unified length of the extended sequence, *h* denotes the filter height. Generally, we set multiple filter heights and adopt a group of filters with a certain height in the Text-CNN for feature extraction. Accordingly, many sets of feature maps are gained by several filters with various heights in the convolutional layer.

Next, The pooling layer carries out further abstraction on feature maps obtained in the convolutional layer. In fact, there are two pooling methods, including average pooling and max pooling. We prefer to capture the most critical local features that significantly affect the classification results, so max pooling is adopted in this layer. Then the fully connected layer takes all of the max pooled features as input and concatenates them together to form high dimensional feature vectors that aggregate all the behavioral semantics information contained in the extended sequences of the input system call traces. The classification layer normalizes the output vectors of the fully connected layer through the softmax function and finally outputs the classification results that indicate the corresponding raw input system call sequences belong to normal or abnormal.

5. Experiments and analysis

5.1. Purposes

In this section, three sets of experiments are conducted to prove the detection performance, accuracy advantage, and robustness against the evolutionary attack of our proposed method. The purposes of these experiments are as follows:

(1) Detection performance evaluation

In the first set of evaluations, to estimate the performance of sequence completion, system call abstraction, and invocation switching differential encoding in the presented method, each experiment is conducted with one operation being applied individually to enhance the behavioral semantics of the raw input system call traces. And all of them are adopted together to test the detection performance in the last evaluation.

(2) Comparison experiments of detection accuracy

To demonstrate the intrusion detection accuracy advantage of the presented method, we compare it with published methods [5, 8,11,13] based on combination rules, n-gram subsequences, and feature matching frequencies in this set of experiments. The Refs. [5,8,11,13] have been introduced in Section 1, and Section 2.

(3) Comparison experiments of obfuscation attack detection

In order to prove the ability and robustness in dealing with the evolutionary attack, comparative experiments among the proposed method and others in Refs. [5,8,11,13] are conducted with taking the obfuscation attack detection into account.

Table 3

Attack types and the number of traces for each type.

No.	Attack type	Attack effect	Attack vector	Number of traces
1	Adduser	Add new superuser	User side infected file	91
2	Hydra_FTP	Password brute force	FTP brute force using hydra	162
3	Hydra_SSH	Password brute force	SSH brute force using hydra	176
4	Java_Meterpreter	Java based meterpreter	Tiki wiki vulnerability exploit	124
5	Meterpreter	Linux meterpreter payload	User side infected file	75
6	Web_Shell	C100 Webshell	PHP remote infected file	118

Table 4

Performance metrics of intrusion detection.

Indices	Description
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$
Recall	$\frac{TP}{TP+FN}$
Precision	$\frac{TP}{TP+FP}$
F1-score	$\frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$
FNR (False Negative Rate)	$\frac{FN}{TP+FN}$
FPR (False Positive Rate)	$\frac{FP}{TN+FP}$

Table 5

Parameters of the system-calls to system-calls model.

Parameters	Value
Epochs	100
Batch_size	128
Rnn_size	50
Num_layers	2
Encoding_embedding_size	20
Decoding_embedding_size	20
Learning_rate	0.001

5.2. Environment

The following experiments are performed on a Windows 10 operation system, powered by Intel(R) Core(TM) i9-9900 CPU @ 3.10 GHz 3.10 GHz and 32.0 GB of RAM. And the TensorFlow version is v1.14.0.

5.3. Data source

We evaluate our approach on the ADFA-LD dataset. It was collected by the ADFA (Australian Defence Force Academy). The ADFA-LD is a recent dataset that contains system call traces and intends to help with the development of host-based intrusion detection systems [37]. Table 2 presents the statistics of this dataset. The dataset has 833 normal traces in TRAINING DATA MASTER (TDM) and 746 abnormal traces in ATTACK DATA MASTER (ADM).

Comparing with other previous benchmark datasets, ADFA-LD is much more representative of current cyber-attacks. It was collected in a real simulation web server environment, and the circumstances details can be found in Serpen et al. [13]. The dataset consists of a comprehensive collection of system call traces representing recent system-level attacks. The attacks used to generate ADFA-LD and the number of collected traces for each type of attack are shown in Table 3. The ADFA-LD dataset has six attack types, including Adduser, Hydra_FTP, Hydra_SSH, Java_Meterpreter, Meterpreter, Web_Shell.

5.4. Evaluation method

The metrics shown in Table 4 are used in the following experiments to evaluate the proposed approach. In this table, TP represents the number of positive (abnormal) system call sequences

Table 6

Parameters of Syscall2Vec, SyscallType2Vec, SyscallCode2Vec.

Parameters	Value
Batch_size	128
Embedding_size	64
Skip_window	1
Num_skips	2
Epoch_num	150,000

Table 7

Parameters of the Text-CNN.

Parameters	Value
Maximum_sequence_length	2948
Embedding_size	64
Filter_size	2,4,6,8,10,12,16,20
Number_of_filters_per_filter_size	128
Batch_size	128
Drop_out	0.3
Learning_rate	0.001
L2_regularization_lambda	0.1

detected as positive (abnormal) sequences, and TN means the number of negative (normal) system call sequences detected as negative (normal) sequences. FP is the number of negative (normal) system call sequences detected as positive (abnormal) sequences, and FN is the number of positive (abnormal) system call sequences detected as negative (normal) sequences.

5.5. Network parameters

The model hyperparameters used in experiments are listed below. These parameters are mainly set by the models' default values [34,38–40] and empirical values in Greff et al. [36].

Table 5 shows parameters and their values of the system-calls to system-calls model adopted in Section 4.2.1. The Rnn_size indicates the number of RNN hidden layer nodes, and the Num_layers indicates the number of stacked RNN cells. The Encoding_embedding_size represents the output contextual information vector's dimension of the encoder, and it is equal to the input vector's dimension of the decoder, i.e., the Decoding_embedding_size. The raw system call traces in the ADFA-LD dataset are used to build the source sequences and the target sequences for training. We construct the training dataset with fixed sequence lengths of 5, 10, 15, 20, 25, and 30, respectively.

The parameters of Syscall2Vec, SyscallType2Vec, and SyscallCode2Vec models mentioned in Section 4.3 are detailed in Table 6. These three models are essentially word2vec, and they share the same parameters in our experiments. The Skip_window is the maximum distance a word can be connected to, the Num_skip refers to the number of samples extracted per target word, and the Embedding_size refers to the dimension of the embedding vector.

The Text-CNN parameters and their values are shown in Table 7. The Maximum_sequence_length refers to the reference value of input length. It is set based on the max sequence length of the ADFA-LD dataset. The Embedding_size indicates the embedding vector dimension of tokens in Section 4.3. The Filter_size

Table 8
Results of the detection performance evaluation.

Experiment	BSEM ^a	Accuracy	Recall	Precious	F1	FNR	FPR
1	None	95.35%	96.92%	93.62%	95.24%	3.08%	6.10%
2	SC ^b	97.67%	96.92%	98.21%	97.56%	3.08%	1.63%
3	SCA ^c	97.25%	97.26%	96.82%	97.04%	2.74%	2.76%
4	DE ^d	97.89%	97.41%	98.26%	97.84%	2.59%	1.66%
5	SC ^b +SCA ^c +DE ^d	98.73%	98.60%	98.60%	98.60%	1.40%	1.16%

^aBehavioral Semantics Enhancement Method.

^bSequence Completion.

^cSystem Call Abstraction.

^dDifferential Encoding.

represents the convolution filter height, which corresponding to a group of filters. To fully extract the local correlation in various scopes, we utilize eight groups of filters with heights from 2 to 20. The Number_of_filters_per_filter_size determines the number of feature maps generated in the convolutional layer, so it is set considering both effective and efficient.

5.6. Procedures

(1) Detection performance evaluation

This set of evaluations contains five independent detection experiments, and the dataset is divided into the training set and testing set by 7:3 in each experiment. In experiment-1, i.e., the baseline test, all of the raw input system call traces in the ADFA-LD dataset are embedded into their representation matrices through Syscall2Vec without using any behavioral semantics enhancement operation proposed in this paper. And then, the matrices are input into a single-channel Text-CNN for training and testing. In experiment-2, only the sequence completion is used to preprocess the raw input system call traces for generating the unified length sequences. And other test settings are the same as in experiment-1. Additionally, the original and predictive behavioral semantics information has already been contained in the unified length sequences, so the single-channel Text-CNN is still used in this evaluation. In experiment-3, we just adopt the system call abstraction to get the type extended sequences. And the Syscall2Vec and SyscallType2Vec are used to generate the representation matrices for the raw sequences and the type extended sequences. In order to incorporate the behavioral semantics information in these two kinds of sequences, we deploy the double-channel Text-CNN in this test. In experiment-4, we take the differential encoding operation singly to generate the code extended sequences. The representation matrices of the raw sequences and the code extended sequences are created via Syscall2Vec and SyscallCode2Vec. The double-channel Text-CNN is utilized to combine the behavioral semantics information of these two kinds of sequences. In the end, we conduct experiment-5 to evaluate our proposed HIDS by applying all of the three behavioral semantics enhancement operations together, and the triple-channel Text-CNN is adopted for feature extraction and data classification.

(2) Comparison experiments of detection accuracy

In this set of experiments, the dataset is divided into the training set and testing set by 7:3. The methods proposed in [5,8,11,13] are used in each experiment independently. The detection indicators and overhead time are recorded for comparison.

(3) Comparison experiments of obfuscation attack detection

For the obfuscation attack detection evaluation, the dataset configuration is that different types (vectors) of attack sample data that belong to the same attack effect in the ADFA-LD dataset need to be put into the training set and testing set, respectively. This setting is more appropriate to evaluate our method's robustness to the evolution attack.

Samples from the two sub-datasets of ADFA-LD (Hydra SSH and Hydra FTP) meet the following two conditions: First, the samples in the two sub-datasets contain different system calls, and the attack sub-sequences in each trace of them are also not the same. Second, the samples of the two sub-datasets indicate the same attack effect of the two different attack vectors. The characteristics of these two sub-datasets are consistent with the definition of evolutionary attack (detailed in Section 3.1). So, in the experiment, we used the Hydra SSH attack samples as the evolutionary samples from Hydra FTP.

The 162 Hydra FTP attack sequences and 162 normal sequences compose the training set, while the 176 Hydra SSH attack sequences and other 176 normal sequences compose the testing set. Next, our approach and the methods presented in Refs. [5,8,11,13] are evaluated on the above dataset, respectively.

5.7. Results

5.7.1. Detection performance evaluation

Table 8 shows the detection results, and Fig. 14 presents the ROC (Receiver Operating Characteristic) curve. Accordingly, two observations can be made from it:

(1) For the first line in Table 8, the performance is already so good with no behavioral semantics enhancement operation in use. This is due to the satisfactory generalization performance of word2vec and Text-CNN in token embedding and feature extraction tasks. It also proves that the viewpoint of analyzing system call sequence from the perspective of behavioral semantic information is appropriate.

(2) The comparison in Table 8 and Fig. 14 between experiment-2, 3, 4, 5, and experiment-1 shows that the detection accuracy and other performance indicators are improved obviously, which verifies the positive role of each operation in the proposed behavioral semantics enhancement method. Furthermore, in experiment-5, all the indices and the ROC curve have reached the optimal, demonstrating the proposed method has a competitive detection performance.

Table 9 presents the overhead time in each test. It shows that Experiment-2 consumes more time than the baseline test. The reason may be that the average sequence length in Experiment-2 is larger than that in the baseline test. While the overhead time increases linearly as the number of channels increases in Experiment-3, 4, and Experiment-5. Because the input data from multiple channels will multiply the workload of the Text-CNN network.

5.7.2. Comparison experiments of detection accuracy

Table 10 gives the detection results, and Fig. 15 presents the ROC curves of different methods. As can be observed from the results, our proposed method is the best among all evaluated approaches as the accuracy is 5.01% to 17.05% higher than other methods. The FNR and FPR metrics are both below 1.50% which are the lowest, while the ROC curve also intuitively shows that

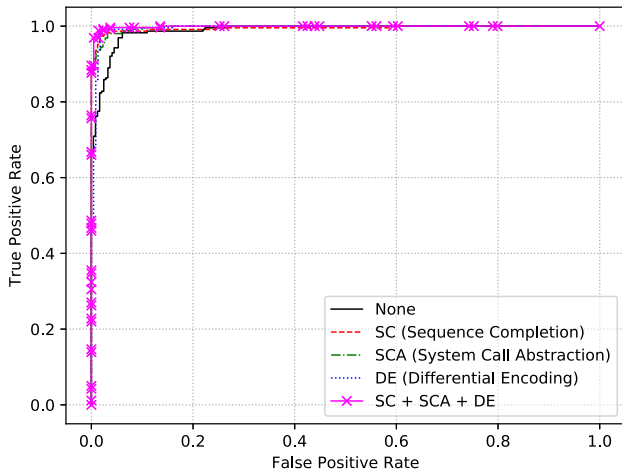


Fig. 14. The ROC curve of detection performance evaluation.

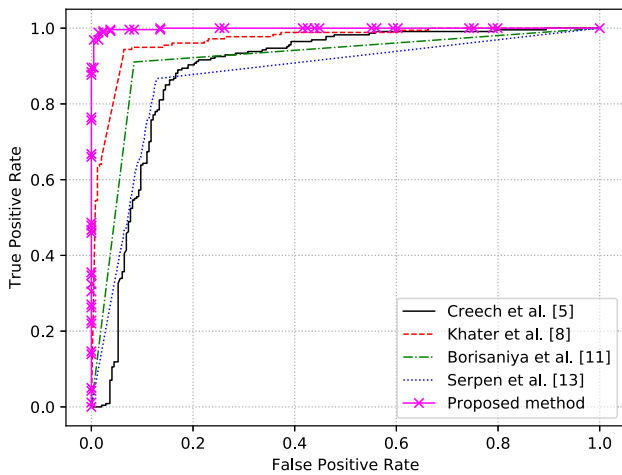


Fig. 15. The ROC curves of detection accuracy comparison experiments.

Table 9

The overhead time of detection performance evaluation.

Experiment	Training time	Testing time	Total running time
1	2929.2325 s	8.6562 s	2940.2577 s
2	2943.6791 s	8.9525 s	2955.6454 s
3	5085.5391 s	15.1498 s	5109.8176 s
4	5062.9457 s	14.9075 s	5085.6031 s
5	9757.8457 s	25.0544 s	9820.3713 s

Table 10

The experiment results of detection accuracy comparison.

Method	Accuracy	Recall	Precious	F1	FNR	FPR
Creech et al. [5]	85.64%	88.55%	82.72%	85.53%	11.45%	17.00%
Khater et al. [8]	93.72%	94.38%	90.81%	92.56%	5.62%	6.75%
Borisaniya et al. [11]	91.35%	91.07%	90.67%	90.87%	8.93%	8.40%
Serpén et al. [13]	81.68%	65.46%	78.23%	71.28%	34.54%	9.69%
Proposed method	98.73%	98.60%	98.60%	98.60%	1.40%	1.16%

our method has the best comprehensive performance. Additionally, other indicators also have significant advantages.

Table 11 displays the overhead time of each experiment. The experimental data shows that our method consumes more training time and testing time. That is related to the deep learning model adopted by our approach. Although the methods in literature [5,8,11,13] have certain advantages in terms of training and testing time, they all require feature engineering with human participation.

Table 11

The overhead time of accuracy comparison experiments.

Method	Training time	Testing time	Total running time
Creech et al. [5]	0.0664 s	0.0270 s	2.1210 s
Khater et al. [8]	0.1141 s	0.0004 s	2.7167 s
Borisaniya et al. [11]	0.8681 s	0.0122 s	12.4579 s
Serpén et al. [13]	0.9909 s	135.1752 s	338.2900 s
Proposed method	9757.8457 s	25.0544 s	9820.3713 s

Table 12

The experiment results of obfuscation attack detection.

Method	Accuracy	Recall	Precious	F1	FNR	FPR
Creech et al. [5]	65.63%	86.36%	61.04%	71.53%	13.64%	55.11%
Khater et al. [8]	85.80%	85.23%	86.21%	85.72%	14.77%	13.64%
Borisaniya et al. [11]	78.69%	88.07%	74.16%	80.52%	11.93%	30.68%
Serpén et al. [13]	60.98%	49.29%	65.60%	56.29%	50.71%	26.87%
Proposed method	95.74%	98.30%	93.51%	95.84%	1.70%	6.82%

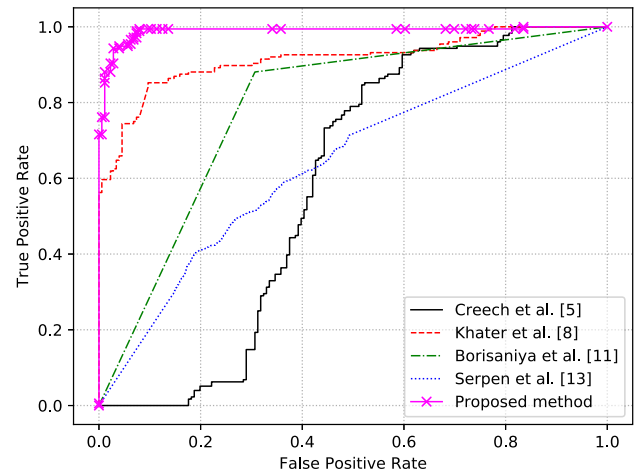


Fig. 16. The ROC curves of obfuscation attack detection experiments.

5.7.3. Comparison experiments of obfuscation attack detection

The experiment results of obfuscation attack detection can be seen in Table 12, and the ROC curve is drawn in Fig. 16. As the results displayed, two observations can be made as follows:

(1) No matter in detection accuracy comparison experiments or obfuscation attack detection evaluations, the presented method performs better than approaches based on combination rules, n-gram subsequences, feature matching frequencies. To be more specific, in this set of experiments, the accuracy of our approach is 9.94% to 34.76% higher than others. And it can be seen from the ROC curve that there is a larger gap between the other methods and the method presented in this paper. These results indicate the proposed method has a more significant advantage in detecting the obfuscation attack.

(2) The robustness of our method is much better than others. The detection performance of our proposed method still has the overall advantages in each indicator, especially the FNR (i.e., 1.70%) is still the lowest and keeps close to that (i.e., 1.40%) of the accuracy comparison experiment. While the performance of other methods reduces obviously, such as the Recall of [13] is below 50%, which have no competitiveness at all. There are reasons to believe that the proposed method is more capable of detecting the evolutionary attack since our approaches can capture more semantic information of malicious behavior.

The overhead time of each experiment is shown in Table 13. Compared with the accuracy comparison experiment, the number of samples in the training set and testing set in this experiment is reduced, so the overhead time used by all methods is decreased.

Table 13

The overhead time of obfuscation attack detection experiments.

Method	Training time	Testing time	Total running time
Creech et al. [5]	0.0394 s	0.0226 s	1.0208 s
Khater et al. [8]	0.0478 s	0.0004 s	2.6943 s
Borisaniya et al. [11]	0.1192 s	0.0052 s	4.0678 s
Serpen et al. [13]	0.1173 s	9.4143 s	24.3258 s
Proposed method	1457.4476 s	16.4580 s	1488.5235 s

Our approach is still the most time-consuming, for reasons given in the previous section.

6. Discussions

Our evaluation results confirm that the implemented HIDS based on the proposed behavioral semantics enhancement method has better detection performance and robustness than these employing features like system call combinations, n-gram subsequences, and feature matching frequencies, especially in detecting evolutionary intrusion such as obfuscation attack. It indicates that the predictive information and high-level behavioral semantic features are more generalized for obfuscation attack detection.

The better performance benefits from the following aspects: First, the sequence completion adopted to regularize system call traces can furthest preserve original features and increase prediction information. It can extend the behavioral semantic information capacity and enlarge the discriminability of different class samples. We do not utilize methods like [13] to truncate the raw sequences by a sliding window of fixed length. This kind of operation will destroy some inherent information, such as consecutiveness features, to a certain extent. Second, the features utilized by [5,8,11], such as system call combinations, n-gram subsequences, and feature matching frequencies, are sensitive to the tampering in system call sequences. In contrast, the system call abstraction and invocation switching differential encoding can improve the abstract representation ability of intrusion behavior. Accordingly, the more stable high-level semantic features can be extracted.

The comparisons in Sections 5.7.2 and 5.7.3 show that the proposed method has no advantages in time consumption, especially the model training time. This paper's HIDS framework uses multi-channel Text-CNN to automatically complete the features fusion, construction, and extraction. It is a ground truth that the hardware resource cost of training the deep learning model is relatively high compared with the general machine learning model. However, we must point out that some manual feature engineering methods may consume more time and labor. For example, the feature dictionary construction process proposed in the literature [5] takes several days, verified when we reproduce this method. Therefore, we still choose Text-CNN to complete the task, resulting from a tradeoff.

On the other hand, the limitations and challenges are as follows. In the currently used ADFA-LD dataset, the sequence elements are only system call numbers. The parameters passed to them are omitted, resulting in the loss of inherent information in this part. However, when various processes invoke the same system call, parameters may be the key to distinguishing the different behaviors. Furthermore, under the current design, when a new kind of extended sequence is generated, the Text-CNN network needs to add an input channel to receive and process the representation matrix of the new extended sequence. However, according to the experimental results in Section 5.7.1, it can be seen that the model training time increases linearly with the increase of the input channels. Therefore, the challenge of high time consumption may rise dramatically.

For future work, we intend to overcome the limitations and challenges. First, we will further study the representation and embedding of system call parameters in the sequence. The emphasis is to analyze and verify the utilization and role of parameters in the classification task of intrusion detection. Second, we plan to study new representation forms, such as graphs, to preserve and merge the information of various extended sequences. Then generate vectors with richer meanings through graph embedding to reduce the input of the Text-CNN network and decrease the processing load.

7. Conclusions

To deal with the challenges of efficient detection against evolutionary host intrusion attacks such as obfuscation technology, we propose a novel behavioral semantics enhancement method of system call sequence for intrusion detection. In this approach, we introduce the idea of behavioral semantics analysis by viewing the system call as a 'word' and the system call sequence as a 'sentence'. This method adopts sequence completion to preserve and enhance the behavioral information of malicious activities and applies system calls abstraction and invocation switching differential encoding to improve the abstractive representation ability of intrusion behavior. The triple-channel Text-CNN that supports three parallel inputs is constructed in our detection system to capture and extract multiple fusion features aiming to achieve the best classification performance. The proposed method is evaluated on the public ADFA-LD dataset experimentally, and the results show that it can detect intrusion attacks more effectively. The comparison experiments prove that our approach can extract more generalized behavioral semantics features, thereby ensuring stability and robustness in detecting obfuscation attacks.

CRedit authorship contribution statement

Yifei Zhang: Conceptualization, Data curation, Investigation, Methodology, Project administration, Software, Writing - original draft, Writing - review & editing. **Senlin Luo:** Conceptualization, Methodology, Project administration, Supervision, Writing - review & editing. **Limin Pan:** Conceptualization, Supervision. **Hanqing Zhang:** Conceptualization, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work is supported in part by the Information Security Software Project (2020) of the Ministry of Industry and Information Technology, PR China, under Grant CEIEC-2020-ZM02-0134.

References

- [1] P. Sharma, J. Sengupta, P. Suri, Survey of intrusion detection techniques and architectures in cloud computing, *Intl. J. High Perform. Comput. Netw.* 13 (2) (2019) 184–198.
- [2] B. Hajimirzaei, N.J. Navimipour, Intrusion detection for cloud computing using neural networks and artificial bee colony optimization algorithm, *ICT Express* 5 (1) (2019) 56–59.
- [3] S. Kalaivani, A. Vikram, G. Gopinath, An effective swarm optimization based intrusion detection classifier system for cloud computing, in: 2019 5th International Conference on Advanced Computing & Communication Systems, ICACCS, 2019.
- [4] M. Liu, Z. Xue, X. Xu, C. Zhong, J. Chen, Host-based intrusion detection system with system calls: Review and future trends, *ACM Comput. Surv.* 51 (5) (2019) 98.

- [5] G. Creech, J. Hu, A semantic approach to host-based intrusion detection systems using contiguous and discontinuous system call patterns, *IEEE Trans. Comput.* 63 (4) (2014) 807–819.
- [6] R. Grishman, *Computational Linguistics: An Introduction*, 1986.
- [7] E. Charniak, *Statistical Language Learning*, 1994.
- [8] B.S. Khater, A.W.B.A. Wahab, M.Y.I.B. Idris, M.A. Hussain, A.A. Ibrahim, A lightweight perceptron-based intrusion detection system for fog computing, *Appl. Sci.* 9 (1) (2019) 178.
- [9] S. Gupta, P. Kumar, An immediate system call sequence based approach for detecting malicious program executions in cloud environment, *Wirel. Pers. Commun.* 81 (1) (2015) 405–425.
- [10] G. Creech, Developing a High-Accuracy Cross Platform Host-Based Intrusion Detection System Capable of Reliably Detecting Zero-Day Attacks (Ph.D. thesis), University of New South Wales, Canberra, Australia, 2014.
- [11] B. Borisaniya, D. Patel, Towards virtual machine introspection based security framework for cloud, *Sadhana-Acad. Proc. Eng. Sci.* 44 (2) (2019) 1–15.
- [12] P. Mishra, A. Negi, E.S. Pilli, R.C. Joshi, VMProtector: Malign process detection for protecting virtual machines in cloud environment, in: *International Conference on Advances in Computing and Data Sciences*, 2019, pp. 360–369.
- [13] G. Serpen, E. Aghaei, Host-based misuse intrusion detection using PCA feature extraction and kNN classification algorithms, *Intell. Data Anal.* 22 (5) (2018) 1101–1114.
- [14] P. Deshpande, S.C. Sharma, S.K. Peddoju, S. Junaid, HIDS: A host based intrusion detection system for cloud computing environment, *Intl. J. Syst. Assur. Eng. Manag.* 9 (3) (2018) 567–576.
- [15] M. Xie, J. Hu, X. Yu, E. Chang, Evaluating host-based anomaly detection systems: Application of the frequency-based algorithms to ADFA-LD, in: *International Conference on Network and System Security*, 2014, pp. 542–549.
- [16] M. Xie, J. Hu, J. Slay, Evaluating host-based anomaly detection systems: Application of the one-class SVM algorithm to ADFA-LD, in: *2014 11th International Conference on Fuzzy Systems and Knowledge Discovery, FSKD*, 2014, pp. 978–982.
- [17] D. Wagner, P. Soto, Mimicry attacks on host-based intrusion detection systems, in: *Proceedings of the 9th ACM Conference on Computer and Communications Security*, 2002, pp. 255–264.
- [18] I. Rosenberg, E. Gudes, Bypassing system calls-based intrusion detection systems, *Concurr. Comput.: Pract. Exper.* 29 (16) (2017).
- [19] I. Bouteraa, M. Derdour, A. Ahmim, Intrusion detection using classification techniques: a comparative study, *Intl. J. Data Mining Modell. Manag.* 12 (1) (2020) 65–86.
- [20] N. Shone, T.N. Ngoc, V.D. Phai, Q. Shi, A deep learning approach to network intrusion detection, *IEEE Trans. Emerg. Topics Comput. Intell.* 2 (1) (2018) 41–50.
- [21] S. Forrest, S.A. Hofmeyr, A. Somayaji, T.A. Longstaff, A sense of self for Unix processes, in: *Proceedings of the 1996 IEEE Symposium on Security and Privacy*, 1996, pp. 120–128.
- [22] S. Forrest, S.A. Hofmeyr, A. Somayaji, *Computer immunology*, *Commun. ACM* 40 (10) (1997) 88–96.
- [23] S.A. Hofmeyr, S. Forrest, A. Somayaji, Intrusion detection using sequences of system calls, *J. Comput. Secur.* 6 (3) (1998) 151–180.
- [24] C. Warrender, S. Forrest, B. Pearlmutter, Detecting intrusions using system calls: alternative data models, in: *Proceedings of the 1999 IEEE Symposium on Security and Privacy*, 1999, pp. 133–145.
- [25] P. Mishra, E.S. Pilli, V. Varadharajan, U. Tupakula, Intrusion detection techniques in cloud environment, *J. Netw. Comput. Appl.* 77 (2017) 18–47.
- [26] Z. Liu, N. Japkowicz, R. Wang, Y. Cai, D. Tang, X. Cai, A statistical pattern based feature extraction method on system call traces for anomaly detection, *Inf. Softw. Technol.* 126 (126) (2020) 106348.
- [27] S. Alarifi, S.D. Wolthusen, Detecting anomalies in IaaS environments through virtual machine host system call analysis, in: *2012 International Conference for Internet Technology and Secured Transactions*, 2012, pp. 211–218.
- [28] T. Mikolov, I. Sutskever, K. Chen, G.S. Corrado, J. Dean, Distributed representations of words and phrases and their compositionality, in: *Advances in Neural Information Processing Systems*, vol. 26, 2013, pp. 3111–3119.
- [29] P. Mishra, I. Verma, S. Gupta, KVMInspector: KVM Based introspection approach to detect malware in cloud environment, *J. Inform. Secur. Appl.* 51 (2020) 102460.
- [30] P. Mishra, V. Varadharajan, E.S. Pilli, U. Tupakula, VMGuard: A VMI-based security architecture for intrusion detection in cloud environment, *IEEE Trans. Cloud Comput.* 8 (3) (2020) 957–971.
- [31] B. Subba, P. Gupta, A tfidfvectorizer and singular value decomposition based host intrusion detection system framework for detecting anomalous system processes, *Comput. Secur.* 100 (2021) 102084.
- [32] I. Sutskever, O. Vinyals, Q.V. Le, Sequence to sequence learning with neural networks, in: *Advances in Neural Information Processing Systems*, vol. 27, 2014, pp. 3104–3112.
- [33] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, Y. Bengio, Learning phrase representations using RNN encoder-decoder for statistical machine translation, in: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP*, 2014, pp. 1724–1734.
- [34] P. Bojanowski, E. Grave, A. Joulin, T. Mikolov, Enriching word vectors with subword information, *Trans. Assoc. Comput. Linguist.* 5 (1) (2017) 135–146.
- [35] J. Wang, C. Zhang, Software reliability prediction using a deep learning model based on the RNN encoder-decoder, *Reliab. Eng. Syst. Saf.* 170 (2018) 73–82.
- [36] K. Greff, R.K. Srivastava, J. Koutnik, B.R. Steunebrink, J. Schmidhuber, LSTM: A search space odyssey, *IEEE Trans. Neural Netw.* 28 (10) (2017) 2222–2232.
- [37] A. Khraisat, I. Gondal, P. Vamplew, J. Kamruzzaman, Survey of intrusion detection systems: techniques, datasets and challenges, *Cybersecurity* 2 (1) (2019) 1–22.
- [38] D. Jatnika, M.A. Bijaksana, A.A. Suryani, Word2Vec Model analysis for semantic similarities in english words, *Procedia Comput. Sci.* 157 (2019) 160–167.
- [39] K.W. Church, Word2vec, *Natural Language Eng.* 23 (1) (2017) 155–162.
- [40] J. Gu, Z. Wang, J. Kuen, L. Ma, A. Shahroudy, B. Shuai, T. Liu, X. Wang, G. Wang, J. Cai, T. Chen, Recent advances in convolutional neural networks, *Pattern Recognit.* 77 (2018) 354–377.
- [41] Y. Kim, Convolutional neural networks for sentence classification, in: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP*, 2014, pp. 1746–1751.



Yifei Zhang received the master's degree from the School of Information Engineering, Communication University of China, Beijing, China, in 2015. He is currently pursuing a doctoral degree at the Information System and Security & Countermeasures Experimental Center, Beijing Institute of Technology. His current research interests include cyberspace security, operating system security, and virtualization security.



Senlin Luo received the bachelor and master degrees from the College of Electrical and Electronic Engineering, Harbin University of Science and Technology, Harbin, China, in 1992 and 1995, respectively, and the doctoral degree from the School of Information and Electronics, Beijing Institute of Technology, Beijing, China, in 1998. He is currently the Deputy Director, the Laboratory Director, and Professor of the Information System and Security & Countermeasures Experimental Center, Beijing Institute of Technology. His current research interests include machine learning, medical data mining, and information security.



Limin Pan received bachelor's and master's degrees from the College of Electrical and Electronic Engineering, Harbin University of Science and Technology, Harbin, China. She is currently working at the Information System and Security & Countermeasures Experimental Center, Beijing Institute of Technology. Her current research interests include machine learning, medical data mining, and information security.



Hanqing Zhang received a bachelor's degree from Harbin Engineering University, Harbin, China, in 2017. He is currently pursuing a doctoral degree at the School of Computer Science and Technology, Beijing Institute of Technology. His current research interests include machine learning, data mining, and mobile security.